SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY

MUKKA, MANGALURU – 574146

A IBM MINIPROJECT REPORT

ON

"LIBRARY MANAGEMENT SYSTEM USING MICROSERVICE ARCHITECTURE"

Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
CYBER SECURITY ENGINEERING

Submitted By,

Khushi Kanchan	01SU23CB015
Manjunath Teli	01SU23CB018

Under the guidance of

Mr.Yogesh

2025-2026

SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY

MUKKA, MANGALURU – 574146

Department of Cyber Security Engineering

CERTIFICATE

This is to certify that the miniproject entitled ***"Library Management System Using Microservice Architecture"*** is a bonafide work carried out by ***Khushi Kanchan, Manjunath Teli*** bearing the USN **01SU23CB015, 01SU23CB018** in the partial fulfillment for the award of **Bachelor of Technology** in **Cyber Security Engineering** of the **Srinivas University, Institute of Engineering & Technology** during the year **2025-2026.** It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the department library. The miniproject report has been approved as it satisfies the academic requirements in respect of miniproject work prescribed for the said degree.

Mr. Yogesh
Mini-project guide

SRINIVAS UNIVERSITY

INSTITUTE OF ENGINEERING & TECHNOLOGY

MUKKA, MANGALURU – 574146

Department of Cyber Security Engineering

DECLARATION

We, **Khushi Kanchan, Manjunath Teli** the students of sixth semester, **B.Tech** in Cyber Security Engineering, Srinivas University, Mukka, hereby declare that the miniproject entitled ***"Library Management System Using MicroService Architecture"*** has been successfully completed by us in partial fulfillment of the requirements for the award of degree in **Bachelor of Technology in Cyber Security Engineering of Srinivas University, Institute of Engineering and Technology** and no part of it has been submitted for the award of degree or diploma in any university or institution previously.

Date: _____

Place: Mukka

ABSTRACT

A Library Management System using microservices architecture is designed to efficiently manage and automate library operations by dividing the application into small, independent, and loosely coupled services. Unlike traditional monolithic systems, this approach improves scalability, flexibility, and maintainability by allowing each service—such as user management, book catalog, issue/return system, and notification service—to function independently and communicate through APIs.

The system enables users to search for books, issue and return resources, and manage accounts seamlessly, while administrators can efficiently handle inventory, track transactions, and monitor user activities. Technologies such as containerization using Docker and deployment through orchestration tools like Kubernetes enhance system reliability and scalability.

By adopting microservices, the system supports continuous integration and deployment, fault isolation, and better resource utilization. This architecture ensures high performance, easier updates, and improved user experience, making it suitable for modern digital libraries and large-scale institutions.

ACKNOWLEDGEMENT

It is our pleasure to acknowledge for all those who have provided guidance, inspiration and encouragement for the successful completion of my mini project. We take this opportunity to express our profound gratitude to our respected guide **Mr.Yogesh**, IBM Professor for his ever inspiring guidance, constant encouragement and support. We sincerely thank **Prof. Gaurav Prasad**, Head of the Department, Cyber Security Engineering, for being an inspiration and support throughout the project. We are extremely grateful to our respected Dean, **Dr. Ramakrishna N Hegde** for providing the facilities to carry out the work. We would like to thank our Management, A. Shama Rao Foundation, Mangalore, for providing the means of support. We also extend my thanks to all teaching, non-teaching staff and management staff of the Cyber Security Engineering Department who have been helpful and cooperative towards the completion of the miniproject.

-Khushi Kanchan

-Manjunath Teli

TABLE OF CONTENTS

Chapter	EVENTSPHERE 2026	Page No.
Chapter 1	INTRODUCTION	1
Chapter 2	OBJECTIVES	2
Chapter 3	SYSTEM ARCHITECTURE	3-4
Chapter 4	SYSTEM DESIGN	5
Chapter 5	IMPLEMENTATION	6
Chapter 6	TECHNOLOGIES USED	7
Chapter 7	WORKING SYSTEM	8
Chapter 8	TESTING AND RESULTS	9
Chapter 9	CONCLUSION	10
Chapter 10	REFERENCES	11

FIGURES

Fig 4.1	Architecture Diagram	5
Fig 4.2	Dataflow Diagram	5
Fig 4.3	Database Design	5
Fig 8.1	SYSTEM DESIGN	5
Fig 8.2	IMPLEMENTATION	6
Fig 8.3	TECHNOLOGIES USED	7

Chapter 1

INTRODUCTION

A Library Management System is an application designed to manage and automate the daily operations of a library, such as maintaining book records, managing user accounts, and handling the issue and return of books. Traditional library systems are often built using monolithic architecture, where all functionalities are tightly integrated into a single system. This approach makes the system difficult to scale, maintain, and update as the number of users and resources increases.

To overcome these challenges, modern systems adopt a microservices architecture, where the application is divided into smaller, independent services that communicate with each other through APIs. Each service is responsible for a specific functionality, such as user management, book catalog management, or transaction handling. This separation allows developers to build, deploy, and scale each service independently, improving system flexibility and performance.

In this project, the Library Management System is implemented using microservices to ensure high scalability, reliability, and maintainability. Technologies like Docker are used for containerization, and orchestration tools like Kubernetes help in managing and deploying services efficiently. The system provides an easy-to-use interface for users and administrators, enabling seamless access to library resources.

This approach not only enhances system performance but also supports continuous integration and deployment, making it suitable for modern digital libraries and large-scale applications.

Chapter 2

OBJECTIVES

The main objectives of developing a Library Management System using microservices are:

1. To automate library operations
   To simplify and digitize tasks such as book management, user registration, and issue/return processes.

2. To implement microservices architecture
   To divide the system into independent services (user service, book service, transaction service) for better modularity and flexibility.

3. To improve scalability
   To allow individual services to scale independently based on demand, ensuring better performance for large numbers of users.

4. To enhance system maintainability
   To make it easier to update, debug, and manage different components without affecting the entire system.

5. To ensure efficient resource utilization
   To optimize system resources by deploying services using tools like Docker.

6. To provide high availability and reliability
   To ensure that failure in one service does not affect the entire system.

7. To enable seamless communication between services
   To use APIs for smooth interaction between different microservices.

8. To improve user experience
   To provide a fast, responsive, and user-friendly interface for both users and administrators.

9. To support continuous integration and deployment
   To enable faster development and updates using modern deployment tools like Kubernetes.

Chapter 3

SYSTEM ARCHITECTURE

1. Overall Architecture

The system follows a client-server architecture with microservices:

- Frontend (Client Layer)
  A user interface (web or mobile application) through which users and administrators interact with the system.
- API Gateway
  Acts as a single entry point for all client requests. It routes requests to the appropriate microservices and handles authentication, logging, and load balancing.
- Microservices Layer
  Contains independent services responsible for specific functionalities.
- Database Layer
  Each microservice manages its own database to ensure loose coupling and data independence.

2. Core Microservices

User Service

- Handles user registration, login, and profile management
- Manages authentication and authorization

Book Catalog Service

- Stores and manages book details (title, author, availability)
- Provides search and filter functionality

Issue/Return Service

- Manages book issuing and returning process
- Tracks due dates and fines

Notification Service

- Sends alerts for due dates, fines, and confirmations
- Can use email or SMS notifications

3. Communication Between Services

- Services communicate using REST APIs or messaging queues
- Ensures loose coupling and independent functioning
- API Gateway routes requests efficiently

4. Deployment Architecture

- Each microservice is containerized using Docker
- Containers are managed and orchestrated using Kubernetes
- Enables:
  - Easy deployment
  - Auto-scaling
  - Fault tolerance

5. Data Management

- Each service has its own database
- Avoids dependency between services
- Improves scalability and data security

6. Key Features of Architecture

- Loose coupling between services
- High scalability
- Fault isolation
- Independent deployment
- Improved performance

7. Architecture Flow (Simple)

1. User sends request from frontend
2. Request goes to API Gateway
3. API Gateway routes to required microservice
4. Microservice processes request and interacts with database
5. Response is sent back to user

Chapter 4

SYSTEM DESIGN

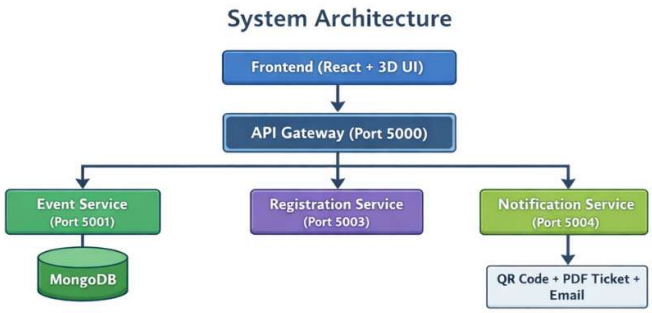


Fig.4.1 Architecture Diagram

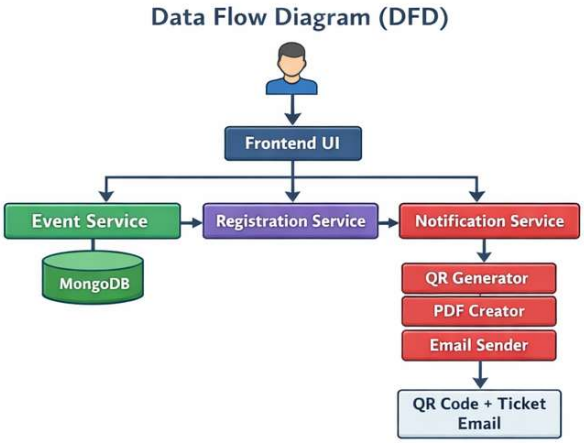
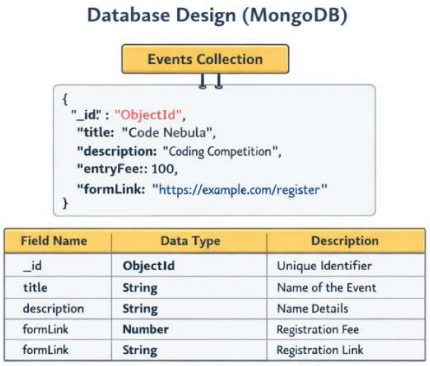


Fig.4.2 Data Flow Diagram



Field Name	Data Type	Description
_id	ObjectId	Unique Identifier
title	String	Name of the Event
description	String	Name Details
formLink	Number	Registration Fee
formLink	String	Registration Link

Fig.4.3 Database Design

Chapter 5

IMPLEMENTATION

The Library Management System using microservices is implemented by dividing the application into multiple independent services, where each service handles a specific functionality such as user management, book catalog, issue/return operations, and notifications, and these services are developed using backend technologies like Node.js or Spring Boot with RESTful APIs to enable communication between them while a user-friendly frontend built using React or Angular interacts with the system through an API Gateway that acts as a single entry point to route requests, handle authentication, and manage logging and validation, and each microservice maintains its own database such as MySQL or MongoDB to ensure data isolation, scalability, and reduced dependency between services, while communication between services is achieved using REST APIs or messaging systems to maintain loose coupling and smooth data exchange, and the deployment and orchestration of services are managed using Kubernetes which provides auto-scaling, load balancing, and fault tolerance, and the system is tested through unit testing and integration testing along with API validation tools to ensure proper functionality, while security is implemented using authentication and authorization mechanisms such as JWT tokens to protect user data and system resources, and overall the workflow involves the user sending a request through the frontend to the API Gateway which forwards it to the appropriate microservice that processes the request, interacts with its database, and returns the response back to the user efficiently.

Chapter 6

TECHNOLOGIES USED

1. Frontend Technologies
   - React.js / Angular
     Used to build an interactive and user-friendly interface for users and administrators.
   - HTML, CSS, JavaScript
     Used for designing and structuring the web pages.
2. Backend Technologies
   - Node.js / Spring Boot
     Used to develop independent microservices and handle server-side logic.
   - Express.js (for Node.js)
     Helps in building RESTful APIs quickly and efficiently.
3. Database
   - MySQL
     Used for structured data storage such as user details and transactions.
   - MongoDB
     Used for flexible and scalable data storage in some services.
4. API Communication
   - REST APIs
     Enable communication between different microservices.
   - JSON
     Used as a data exchange format between client and server.
5. Tools and Platforms
   - Docker
     Used for containerizing microservices to ensure consistency across environments.
   - Kubernetes
     Used for deployment, scaling, and management of containerized applications.
6. API Gateway
   - Express Gateway / Spring Cloud Gateway
     Acts as a single entry point for all client requests and routes them to appropriate services.
7. Testing Tools
   - Postman
     Used for testing APIs and verifying endpoints.
8. Version Control
   - Git & GitHub
     Used for source code management and collaboration.
9. Security Technologies
   - JWT (JSON Web Token)
     Used for authentication and authorization.

.

Chapter 7

WORKING SYSTEM

User Interaction
The user (student/admin) accesses the system through a web or mobile interface and performs actions such as login, searching for books, or issuing/returning books.

Request to API Gateway
The request from the frontend is sent to the API Gateway, which acts as the single entry point for all operations.

Authentication Process
The API Gateway forwards the request to the User Service for authentication and authorization using secure methods like JWT.

Service Routing
Based on the request type, the API Gateway routes it to the appropriate microservice:

- Book Catalog Service → for searching books
- Issue/Return Service → for borrowing or returning books
- User Service → for account-related operations

Processing by Microservices
The respective microservice processes the request and performs the required operation by interacting with its own database.

Database Interaction
Each microservice accesses its dedicated database (MySQL/MongoDB) to retrieve or store data without affecting other services.

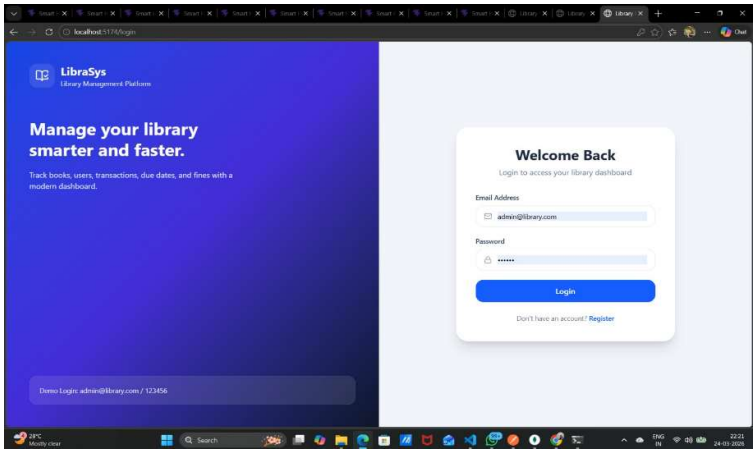
Inter-Service Communication
If needed, microservices communicate with each other using REST APIs to complete complex operations.

Notification Handling
The Notification Service sends alerts such as due dates, confirmations, or fine reminders to users.

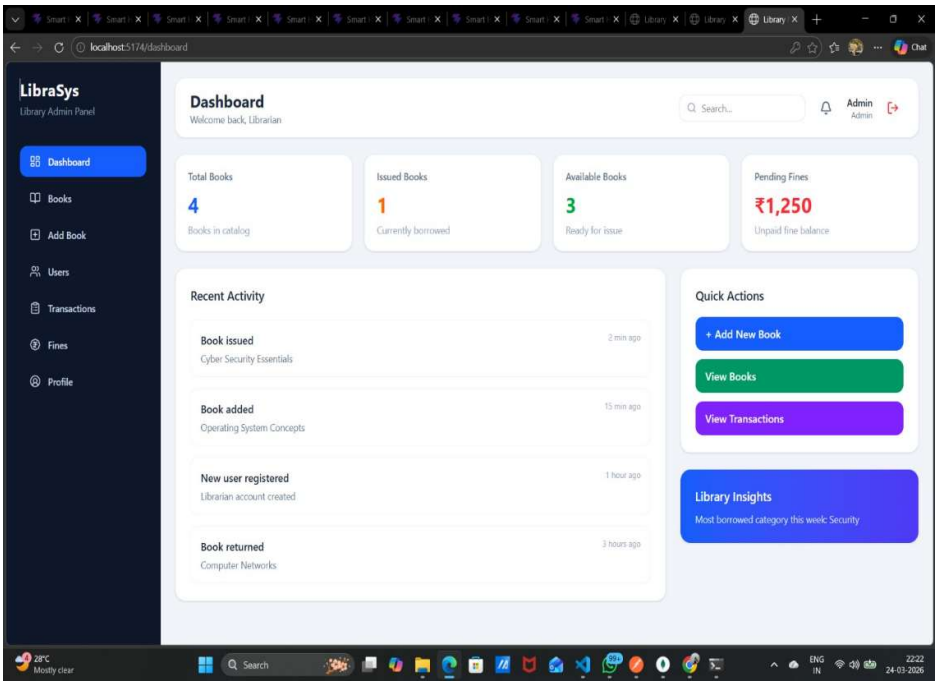
Response to Client
After processing, the response is sent back through the API Gateway to the frontend and displayed to the user.

Chapter 8

TESTING & RESULTS



.



Chapter 9

CONCLUSION

The Library Management System using microservices provides an efficient, scalable, and flexible solution for managing modern library operations by dividing the application into independent services such as user management, book catalog, and transaction handling, which improves system performance, maintainability, and fault isolation compared to traditional monolithic systems, and by enabling seamless communication through APIs and supporting independent deployment of services, the system ensures better resource utilization and reliability, while the use of modern tools like Kubernetes enhances deployment, scalability, and fault tolerance, making the system suitable for large-scale applications and digital libraries, and overall this approach results in a user-friendly, secure, and high-performing system that meets the needs of both users and administrators efficiently.

Chapter 10

REFERENCES

- **Kubernetes Documentation – https://kubernetes.io/docs/**
- **Node.js Official Documentation – https://nodejs.org/**
- **Spring Boot Documentation – https://spring.io/projects/spring-boot**
- **MongoDB Documentation – https://www.mongodb.com/docs/**
- **MySQL Documentation – https://dev.mysql.com/doc/**
- **React Documentation – https://react.dev/**
- **Express.js Documentation – https://expressjs.com/**
- **Newman, Sam – *Building Microservices* (Book)**
- **Richardson, Chris – *Microservices Patterns* (Book)**